

Java Streams

Interview Practice Set — Questions, Solutions & Explanations

The dataset used throughout

```
record Employee(String name, String dept, int salary, int age) {}

List<Employee> employees = List.of(
    new Employee("Anna", "Engineering", 95000, 29),
    new Employee("Bob", "Engineering", 70000, 35),
    new Employee("Charlie", "Sales", 60000, 41),
    new Employee("Diana", "Sales", 82000, 38),
    new Employee("Eve", "HR", 55000, 26),
    new Employee("Frank", "Engineering", 88000, 45)
);
```

Tip: try each yourself before reading the solution. The explanation matters more than the code — interviews test whether you can talk through it.

Section 1 — groupingBy in depth

Q1. groupingBy + mapping

Build a `Map<String, List<String>>` mapping each department to the list of employee NAMES in it (not the whole objects).

Solution

```
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.mapping(Employee::name, Collectors.toList())));
```

Why / interview angle

The downstream of `groupingBy` works on whole `Employee` objects. To turn them into names AND then collect, you nest `mapping(transform, thenCollector)`. Read it: “group by dept; within each bucket, map each employee to a name, then collect into a list.” This “collector inside a collector” idea is the single most-tested grouping pattern beyond the basics.

Q2. groupingBy with a computed key

Group employees into salary bands: key “high” if salary > 75000, else “low”. Return `Map<String, List<Employee>>`.

Solution

```
Map<String, List<Employee>> bands = employees.stream()
    .collect(Collectors.groupingBy(
        e -> e.salary() > 75000 ? "high" : "low"));
```

Why / interview angle

The classifier is ANY function, not just a getter. Here it computes the key on the fly. Interviewers use this to check you understand the classifier is just “element → key”. Common variants: group by age decade with `e -> e.age() / 10 * 10`.

Q3. groupingBy + maxBy (highest paid per dept)

Build a Map<String, Optional<Employee>> giving the highest-paid employee in each department.

Solution

```
Map<String, Optional<Employee>> topByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        Collectors.maxBy(Comparator.comparingInt(Employee::salary))));
```

Why / interview angle

maxBy returns an Optional because a bucket could theoretically be empty. If asked to avoid the Optional wrapper, use collectingAndThen(maxBy(...), Optional::get) — that unwraps it after collecting. Knowing collectingAndThen impresses interviewers.

Q4. Sorted keys (TreeMap)

Same as Q1 in Section 1 (count per dept) but with keys sorted alphabetically. Map<String, Long>.

Solution

```
Map<String, Long> sortedCounts = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::dept,
        TreeMap::new,
        Collectors.counting()));
```

Why / interview angle

The three-arg groupingBy adds a map factory in the middle. TreeMap::new gives sorted keys; LinkedHashMap::new preserves insertion order. Classic interview question: “how do you get the grouped result sorted by key?” — answer is the middle argument.

Section 2 — partitioningBy

`partitioningBy` is a special case of `groupingBy` where the key is a boolean. It ALWAYS returns exactly two keys — `true` and `false` — even if one side is empty. That guaranteed-two-buckets behavior is the difference interviewers want you to state.

Q5. partitioningBy basics

Split employees into those earning > 75000 (`true`) and the rest (`false`). `Map<Boolean, List<Employee>>`.

Solution

```
Map<Boolean, List<Employee>> parts = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.salary() > 75000));

List<Employee> highEarners = parts.get(true);
List<Employee> rest       = parts.get(false);
```

Why / interview angle

`groupingBy` with the same boolean classifier would also work, BUT if no employee matched, `groupingBy` would omit that key entirely, while `partitioningBy` still gives you an empty list under it. That guarantee is the whole point. It also takes a downstream: `partitioningBy(pred, counting())`.

Section 3 — toMap (and the duplicate-key trap)

Q6. toMap basics

Build a `Map<String, Integer>` from employee name to salary.

Solution

```
Map<String, Integer> nameToSalary = employees.stream()
    .collect(Collectors.toMap(Employee::name, Employee::salary));
```

Why / interview angle

`toMap` takes two functions: one for the key, one for the value. `Function.identity()` as the value keeps the whole object. This works here because names are unique.

Q7. toMap duplicate-key trap — THE classic

Build a `Map<String, Integer>` from department to salary. What goes wrong, and how do you fix it?

Solution

```
// This THROWS IllegalStateException – duplicate key "Engineering":
employees.stream()
    .collect(Collectors.toMap(Employee::dept, Employee::salary));

// Fix: supply a merge function for collisions
Map<String, Integer> totalByDept = employees.stream()
    .collect(Collectors.toMap(
        Employee::dept,
        Employee::salary,
        (a, b) -> a + b)); // keep sum; or (a,b)->a to keep first
```

Why / interview angle

This is one of the most asked stream questions. Without the third argument, two employees in the same dept produce duplicate keys and `toMap` throws `IllegalStateException`. The merge function `(a, b) -> ...` decides

who wins: sum them, keep the first $((a, b) \rightarrow a)$, or the last $((a, b) \rightarrow b)$. A 4th argument (map supplier) lets you pick the map type, e.g. `TreeMap::new`.

Section 4 — flatMap

map is one-to-one (each element → one element). flatMap is one-to-many, then flattens all the little streams into a single stream. Use it whenever each element itself contains a collection you want to “unpack”.

Q8. flatMap to unpack nested lists

Given List<List<Integer>> nested, produce one flat List<Integer> with duplicates removed.

Solution

```
List<List<Integer>> nested = List.of(
    List.of(1, 2, 3), List.of(2, 4), List.of(5));

List<Integer> flat = nested.stream()
    .flatMap(List::stream) // each inner list -> a stream, all merged
    .distinct()
    .toList();             // [1, 2, 3, 4, 5]
```

Why / interview angle

The key line is flatMap(List::stream): each inner List becomes a Stream, and flatMap merges them into one. If you used map(List::stream) you'd get a useless Stream<Stream<Integer>>. State the difference out loud: “map keeps the nesting, flatMap removes one level.”

Q9. flatMap on split strings

Given List<String> sentences, produce a List<String> of all individual words.

Solution

```
List<String> sentences = List.of("hello world", "java streams");

List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .toList();      // [hello, world, java, streams]
```

Why / interview angle

Each sentence splits into an array, Arrays.stream(...) turns it into a stream, and flatMap merges them. This “words from sentences” task is a favorite flatMap interview prompt.

Section 5 — reduce

reduce folds a stream down to a single value by repeatedly combining elements. Three forms exist; interviewers most often ask you to explain the identity and the accumulator.

Q10. reduce — sum with identity

Sum all salaries using reduce (not sum()).

Solution

```
int total = employees.stream()
    .mapToInt(Employee::salary) // IntStream
    .reduce(0, Integer::sum);    // identity 0, accumulator +

// On a Stream<Integer> the same idea:
int total2 = employees.stream()
    .map(Employee::salary)
```

```
.reduce(0, Integer::sum);
```

Why / interview angle

The identity (0) is the starting value AND what's returned for an empty stream — that's why sum's identity is 0 and product's would be 1. The accumulator (a, b) -> a + b combines the running result with the next element. With NO identity, reduce returns an Optional (empty stream → nothing to return).

Q11. reduce — build a string

Concatenate all names into one comma-separated string using reduce, then note the better tool.

Solution

```
String joined = employees.stream()
    .map(Employee::name)
    .reduce("", (a, b) -> a.isEmpty() ? b : a + ", " + b);

// Better in practice – use joining:
String better = employees.stream()
    .map(Employee::name)
    .collect(Collectors.joining(", "));
```

Why / interview angle

Good to show you CAN do it with reduce, but say the smart part: for strings, Collectors.joining is cleaner and avoids repeated string concatenation. Knowing when NOT to use reduce is itself a signal of maturity.

Section 6 — Laziness & “trace the output”

These puzzles test whether you understand that intermediate operations are lazy and elements flow through the pipeline ONE AT A TIME, not stage by stage.

Q12. What does this print?

Predict the exact output.

Solution

```
Stream.of("a", "b", "c")
    .filter(s -> { System.out.println("filter: " + s); return true; })
    .map(s -> { System.out.println("map: " + s); return s.toUpperCase(); })
    .findFirst();
```

Why / interview angle

Output is just two lines: filter: a then map: a. Only “a” flows all the way through because findFirst short-circuits after the first result. The insight: elements are pulled one at a time through the WHOLE pipeline, not “filter everything, then map everything.” This element-by-element model is the #1 laziness concept tested.

Q13. Nothing runs without a terminal op

What does this print?

Solution

```
Stream.of(1, 2, 3)
    .map(n -> { System.out.println("mapping " + n); return n * 2; });
// (no terminal operation)
```

Why / interview angle

Nothing prints. Intermediate operations are lazy — with no terminal operation (collect, forEach, count...), the pipeline never executes. Interviewers love this: “Why does my map not run?” Answer: you never added a terminal operation.

Q14. Stream is single-use

What happens here?

Solution

```
Stream<Integer> s = Stream.of(1, 2, 3);
s.forEach(System.out::println); // works
s.forEach(System.out::println); // throws!
```

Why / interview angle

The second call throws `IllegalStateException: stream has already been operated upon or closed`. A stream is consumed by its terminal op and cannot be reused — create a fresh one from the source. State this crisply; it's a common quick-fire question.

Section 7 — Verbal questions (say these out loud)

Interviews test explaining as much as coding. Practice answering each in 1–2 sentences.

1. What is a stream?

A pipeline for processing a sequence of elements. It doesn't store data; it carries elements from a source through operations to a result. It doesn't modify the source.

2. Intermediate vs terminal operation?

Intermediate ops (filter, map, sorted) are lazy and return a stream. Terminal ops (collect, count, forEach) trigger execution and produce a result. Nothing runs until a terminal op is present.

3. What does “lazy” mean?

Intermediate operations don't execute until a terminal op runs. This enables short-circuiting (findFirst, limit, anyMatch) and avoids unnecessary work.

4. map vs flatMap?

map is one-to-one; flatMap is one-to-many that flattens the resulting streams into a single stream. Use flatMap to unpack nested collections.

5. Why does max/findFirst return Optional?

Because the stream might be empty — Optional represents “maybe a value.” Unwrap safely with orElse / orElseThrow / ifPresent rather than .get().

6. Why is .get() on Optional risky?

It throws NoSuchElementException if the Optional is empty — reintroducing the crash Optional was meant to prevent. Prefer orElse, orElseThrow, or ifPresent.

7. Can you reuse a stream?

No. After a terminal op the stream is consumed; reusing it throws IllegalStateException. Create a new stream from the source.

8. toMap with duplicate keys?

Throws IllegalStateException unless you provide a merge function as the third argument to resolve collisions.

9. findFirst vs findAny?

findFirst respects encounter order; findAny may return any element and is cheaper in parallel streams.

10. Are streams faster than loops?

Not inherently — often slightly slower for simple cases. Streams optimize readability and expressiveness, not raw speed.

11. When would you NOT use a stream?

Complex logic with side effects, early returns/break, heavy debugging, or when a plain loop is simply clearer. “Always streams” is a red flag.

12. When do parallel streams help / hurt?

Help: large datasets, CPU-bound, stateless, associative operations. Hurt: small data, ordered dependencies, shared mutable state, or I/O-bound work. Always benchmark.

13. == vs .equals() for strings?

== compares references; .equals() compares content. Always use .equals() (or put the constant first: "X".equals(v)) to be null-safe.

14. .toList() vs collect(Collectors.toList())?

.toList() (Java 16+) returns an immutable list; collect(toList()) usually returns a mutable ArrayList. Adding to an immutable list throws UnsupportedOperationException.

Study order: work Sections 1–6 with code in hand, then drill Section 7 out loud until each answer is automatic. When all of this feels easy, you're interview-ready on streams.